

Ejemplo

- Vamos a usar como ejemplo el problema de localización-asignación que vimos en el tema de Relajación lagrangeana.
- Recordemos que el modelo usa dos tipos de variables:
- y_i , binaria que indica si se abre el centro en i .
- x_{ij} , binaria que indica si el cliente j se asigna al centro i .
- El modelo era:

$$\begin{aligned} \min \quad & \sum_{i=1}^m f_i y_i + \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} \\ \text{s.a.} \quad & \sum_{i=1}^m x_{ij} = 1, & j = 1, \dots, n & \quad (1) \end{aligned}$$

$$\begin{aligned} & \sum_{j=1}^n x_{ij} \leq n y_i, & i = 1, \dots, m & \quad (2) \\ & y_i, x_{ij} \in \{0, 1\} & i = 1, \dots, m; j = 1, \dots, n \end{aligned}$$

Ejemplo

- ¿Qué será una solución “vecina” en este problema?

Ejemplo

- Pero podemos hacer algo un poco más estructurado:
- Notemos que, **con** y **fijos**, el problema en los x es un problema “fácil”.
- Sea

$$g(y) = \min \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$
$$\text{s.a.} \quad \sum_{i=1}^m x_{ij} = 1, \quad j = 1, \dots, n \quad (1)$$

$$\sum_{j=1}^n x_{ij} \leq n y_i, \quad i = 1, \dots, m \quad (2)$$
$$x_{ij} \in \{0, 1\} \quad i = 1, \dots, m; j = 1, \dots, n$$

- Luego, podemos hacer la búsqueda sólo en los y con función objetivo

$$f(y) = \sum_{i=1}^m f_i y_i + g(y)$$

y la evaluación de esto es barata...

- Veamos esto funcionando bajo distintas situaciones...

Ejemplo

- Se ve que el SA llega a una solución bastante buena...
- Pero igual se puede obtener la solución óptima en poco tiempo
- Pero: un caso con $n = 80$ posibles centros, $m = 2000$ clientes, llega a gap de 0,58% recién después de 2 horas y 20 minutos.
- Mejor valor de solución (incumbente): 6169.
- Mejor cota: 6136,3
- Veamos el SA para este problema...

Tabu Search (Búsqueda Tabú)

- Este procedimiento trata de resolver los problemas que presentan los métodos anteriores, con varios enfoques:
- Siempre hay riesgo de volver a una solución ya visitada.
- Para eso vamos a “prohibir” visitar soluciones que ya han sido analizadas.
- Para esto, guardamos las soluciones en una “Lista Tabú”
- Además, también vamos a permitir ir a soluciones peores.
- En algunos momentos vamos a cambiar completamente la estructura de las soluciones que se están analizando: Esto se llama “diversificación”.

Tabu Search (Búsqueda Tabú)

- El algoritmo mantiene una lista, L , de soluciones “tabú”
- **Tabu Search básico:**
 0. Sea $x^0 \in C, k = 0, L = \{\emptyset\}$.
 1. **Etapas k:** Estamos en x^k y se verifica criterio de parada.
 2. Si se continúa, construir vecindad $V(x^k)$.
 3. Elegir y solución de $\min\{f(x) : x \in V(x^k)\}$.
 4. Si $y \in L$, elegir otro $y : f(y) < f(x^k), y \notin L$
 5. Si no se puede encontrar y adecuado, hacer uno de los siguientes:
 - 5.1. Aceptar $y : f(y) > f(x^k)$ con alguna probabilidad (“estilo simulated annealing”).
 - 5.2. Aplicar **DIVERSIFICACION** para generar un nuevo y .
 6. $x^{k+1} = y, k = k + 1, L = L \cup \{y\}$, ir a 1.

Tabu Search: Comentarios

- El procedimiento fue desarrollado por Fred Glover en los 70.
- La construcción de $V(x^k)$ se hace, muchas veces, en forma aleatoria y la idea es que sea “simple”.
- Notar que el procedimiento hace una optimización local.
- Uno de los aspectos claves es el largo de la lista Tabú, el que se calibra usando simulación.
- Cuando se acepta una solución peor, se puede hacer “al estilo Simmulated Annealing”, entre otras.
- **DIVERSIFICACION** es una etapa importante: se trata de generar un patrón de solución completamente diferente, es decir, “saltar” a otra región del espacio de búsqueda.
- La Búsqueda Tabú ha tenido éxito aplicada a algunos problemas combinatoriales de gran tamaño, por ejemplo, problemas de ruteo vehicular.

Algoritmos Genéticos

- Un conjunto completo de métodos heurísticos se ha clasificado bajo el nombre de “Algoritmos Evolutivos” o “Algoritmos de población”.
- La idea es imitar procesos evolutivos o biológicos.
- La primera etapa es codificar las soluciones factibles del problema en los siguientes elementos:
 - ▶ Una conjunto G de “genes”.
 - ▶ Cada solución factible del problema combinatorial es representada por un elemento $(g_1, g_2, \dots, g_p)$, donde $g_i \in G$. Este es un “cromosoma”.
 - ▶ Se define una función $h(g_1, g_2, \dots, g_p)$, llamada “función de adaptación” (“fitness function”).

Algoritmos Genéticos

- **Algoritmo Genético General:**

0. Comenzar con una población $U^0 \subset G^p$.
1. Elegir un conjunto $B \subset U$ de individuos “mejor adaptados”, usando para ello la función h .
2. Aplicar “operadores genéticos” a B para generar un nuevo conjunto B' con mejores individuos que B .
3. B' reemplaza a B en U .
4. Verificar criterio de parada, STOP o ir a 1.

- Elemento central es la definición de los **Operadores Genéticos**:

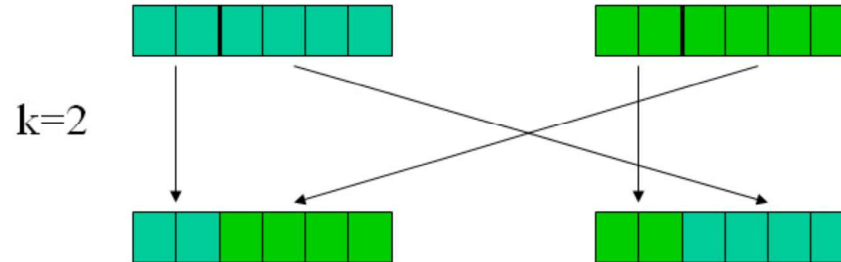
Algoritmos Genéticos

- **Mutación:**

- Sea $v = (g_1, \dots, g_p) \in G^p$ un cromosoma.
- Elegir $j \in \{1, \dots, p\}$ aleatoriamente.
- Reemplazar g_j con $g' \in G, g' \neq g_j$.
- Esto genera un nuevo individuo cuya “adaptabilidad” es evaluada usando h .
- Si es mejor que el anterior, se conserva. Si no, se busca otra mutación.

Algoritmos Genéticos

- **Crossover (“recombinación”):**
- Sean $v = (v_1, \dots, v_p) \in G^p$ y $w = (w_1, \dots, w_p) \in G^p$ dos cromosomas.
- Elegir $k \in \{1, \dots, p\}$ aleatoriamente.
- Generamos dos nuevos individuos, v' y w' :
 - ▶ $v' = (w_1, \dots, w_k, v_{k+1}, \dots, v_p)$
 - ▶ $w' = (v_1, \dots, v_k, w_{k+1}, \dots, w_p)$



- Conservamos los nuevos individuos si están “mejor adaptados”.

Algoritmos Genéticos: Comentarios

- Preguntas claves:
 - ¿Cómo seleccionamos la población inicial?
 - ¿Cómo elegimos los individuos para Crossovers y Mutación?
- También es muy importante la función de adaptabilidad.
 - ▶ Podría usarse la función objetivo original...
 - ▶ Pero también puede incluir una medida de satisfacción de restricciones...
- Los AG se han aplicado exitosamente en algunos problemas relacionados a secuenciamiento (“scheduling”).

Ejemplo

- Tomemos nuevamente nuestro problema de Localización y Asignación de clientes.
- Un Algoritmo Genético se puede desarrollar de muchas formas diferentes.
- Notemos, eso sí, que la codificación es “natural” ya que tenemos variables 0-1
- Tomemos el mismo esquema para el problema que usamos en el ejemplo de Búsqueda Tabú:
- El cromosoma es una solución $y = (y_1, \dots, y_m)$ y los genes son los valores y_i .
- Como función de adaptación podemos tomar:

$$h(y) = - \left(\sum_{i=1}^m f_i y_i + g(y) \right)$$

- La población inicial: podemos generar un conjunto de “cromosomas”, en forma aleatoria
- Luego, usar los operadores de mutación y recombinación.

Otras Ideas

- Existen variantes de las anteriores para problemas especiales.
- Existen también otras ideas:
- Enfoques de “poblaciones colaborativas”:
- Algoritmos de “Colonias de Hormigas” (Ant Colony), de “abejas”, de “ballenas” y de otros animales...
- Simula el comportamiento colaborativo para resolver un problema difícil.
- Hay transmisión de información mediante “feromonas”

Comentarios Finales

- ¿Cuál es la mejor heurística para un problema?
- ... Depende....
- El buen funcionamiento depende mucho de la calibración adecuada de parámetros.
- Pero hay mucha experiencia práctica que sirve de guía.
- Por ejemplo, la Búsqueda Tabú ha sido muy buena en problemas “tipo” PVV, ruteo de vehículos y relacionados.
- Existen gran cantidad de estudio teóricos sobre ciertos procedimientos heurísticos para saber qué tan buenos son.
- Un punto central: balance entre velocidad de resolución y calidad de la solución.